المرحلة الثانية — ربط الواجهة والكارت والسيكشنات بقواعد العرض الموحدة

الملف المخصص للزميل الثاني
شرط البدء: لا تبدأ قبل استلام contract المرحلة الأولى، خصوصًا تعريف `globalOffer` بين متاجر المنصة، شرط مالك العرض في فلتر المتجر، و `store_product_id`.

الهدف

جعل كل ظهور للمنتج مطابقًا للفلتر والسياق التجاري:

- المنصة العامة وكارت البراند: الأرخص بين متاجر منصة Élan Scents فقط لنفس العطر ونفس الحجم.
- فلتر متجر محدد وصفحة المتجر: تعرض نفس العرض الأرخص فقط إذا كان المتجر المحدد هو صاحبه، لا يوجد سعر متجر بديل.
- السيكشن التلقائي المرتبط بمتجر: لا يعرض منتجًا بسبب عرض عام خارج المتجر.
- الكارت: يحتفظ بالعرض الذي اختاره العميل ولا يعيد استبداله تلقائيًا بعرض متجر آخر.
- التقاطع بين الجنس/المناسبة/النوتة/البراند/الحجم/السعر: AND بين الأبعاد المختلفة.

1. توحيد helpers في `app-core.js`

استبدل الاستدعاءات الغامضة لـ `best(p)` بعقدة صريحة، مثل:

```
globalOffer(p)              // cheapest eligible offer among platform stor
storeOwnsGlobalOffer(p, storeId) // filter predicate only
offerForContext(p, ctx)     // always returns globalOffer
```

`list()` يجب أن يحدد context مرة واحدة:

```
ctx.storeId = F.st || null
ctx.offer = globalOffer(p)
ctx.storeMatch = !ctx.storeId || storeOwnsGlobalOffer(p, ctx.storeId)
```

ثم يطبق:

1. البحث والبراند.
2. الجنس/الفئة/المناسبة/النوتة والحجم.
3. وجود `ctx.offer` ونجاح `ctx.storeMatch`.
4. min/max على `ctx.offer.price`.
5. flags على العرض/الحجم وفق contract المرحلة الأولى.

لا تستخدم فحصًا من نوع `p.offers.some(store == selectedStore)` ثم تعرض `x.b` العام؛ هذا يعيد الخلل بصورة أخرى.

2. بطاقة المنتج والصفحات

عدّل `allSectionCards`, `topSectionCards()`, `prodView()`, `pc()` `()` وأي مكان يبني `{p,b}`:

- مرّر `offer` الناتج من context إلى البطاقة.
- البطاقة العامة تعرض store/price العام.
- البطاقة في صفحة المتجر تعرض نفس store/price الأساسي، بشرط نجاح `ctx.storeMatch`.
- صفحة المنتج العامة تعرض السعر العام فقط.
- صفحة المنتج داخل store context تعرض السعر الأساسي نفسه عند الضغط على Add.
- عند تعدد الأحجام، كل زر حجم يحسب `globalOffer` لنفس الحجم ويتحقق من مالك المتجر، ولا يستدعي سعر متجر خاص.
- لا تخلط بين `product_id` و`product_size_id` في روابط التنقل أو `data-add`.

أضف indicator اختياريًا في التطوير يوضح `price`, `storeProductId`, `storeId`, لتسهيل اكتشاف التعارض، ثم اخفه في الإنتاج.

3. الكارت — أهم نقطة في المرحلة الثانية

المشكلة الحالية أن الكارت يخزن `sid` ثم يعيد استدعاء `best(p)` في `cartHTML`/`cartGroups()` `()` /الطلب.
المطلوب:

```
CART item = {
  sid: product_size_id,
  storeProductId: selected_offer.storeProductId,
  storeId: selected_offer.storeId,
  observedPrice: selected_offer.price,
  currency: selected_offer.cur,
  q: quantity
}
```

قواعد التنفيذ:

1. Add من الصفحة العامة يخزن `globalOffer.storeProductId` .
2. Add من فلتر/صفحة متجر يخزن نفس `globalOffer.storeProductId` بعد اجتياز شرط مالك المتجر.
3. cart grouping يعتمد على `storeId` المخزن، لا على إعادة حساب السعر.
4. السعر المعروض في الكارت يأتي من العرض الأساسي المخزن/المتحقق، مع refresh عند تحميل catalog.
5. إذا اختفى العرض أو تغير السعر، لا تنقل العنصر بصمت إلى متجر آخر؛ اعرض حالة تحتاج مراجعة أو أعد اختيار العرض.
6. `sendStoreOrder()` يرسل `storeProductId` الصحيح لكل line، والخادم يظل مصدر التحقق النهائي.
7. عند إعادة المزامنة أو تسجيل الدخول، لا تدمج نفس `product_size_id` من متجرين في عنصر واحد بلا قرار واضح.
8. preserve backward compatibility للعناصر القديمة التي تحتوي `sid` فقط: اعتبرها global مرة واحدة، ثم اكتب الشكل الجديد.

السطر في الكارت هو: `product_size_id + store_product_id + quantity + observed_price`. عند فتح الكارت أو تحديث الكتالوج، يتحقق الخادم من العرض الأساسي الحالي. إذا تغير السعر أو المخزون أو التوفر أو توقف المتجر، لا تنقل السطر بصمت إلى متجر آخر؛ اعرض سبب التغيير واطلب قبول التحديث قبل إعادة التسعير. وعند إنشاء الطلب، يعيد الخادم التحقق داخل معاملة واحدة من `store_product_id` والسعر والعملة والكمية.

4. `design-layer.js`

في `autoList()` :

- استبدل مقارنة الاسم `sameCatalogName(o.store, f.store)` بمعرف متجر إذا كان contract يدعمه.
- استخدم `globalOffer` في كل المسارات؛ `allEligibleOffers` للتحقق فقط.
- مرّر العرض الذي تم اختياره إلى `pc()` بدل إعادة حسابه.
- راجع `manualList()` : يعرض `globalOffer` ، وإذا كان السيكشن يحمل store filter يرشح فقط صاحب `globalOffer` .

- راجع flags بحيث لا يحصل عرض متجر على flag لا ينطبق عليه.
- امنع إظهار عنصر يدوي غير متاح، أو بيّن أن manual selection يتجاوز الفلترة التجارية إذا كان هذا قرار المنتج.

5. الفلاتر الثلاثة المطلوبة

قاعدة التنفيذ المشتركة

لا تحول `occasion_id` , `category_id` , أو `note_id` إلى رقم ترتيب داخل `texts.js` . يجب أن تكون قيمة select هي UUID أو `key` الثابت القادم من المرحلة الأولى، بينما label العربي/الإنجليزي للعرض فقط.

عند بناء النتيجة:

```
gender = products.gender
categoryIds = product_categories.category_id
occasionIds = product_occasions.occasion_id
noteIds = product_notes.note_id
```

طبّق AND بين gender/category/occasion/note/brand، وOR داخل مجموعة القيم لنفس البعد عند دعم التعدد. لا تستخدم اسمًا مترجمًا للمقارنة، ولا تسقط العلاقة بصمت إذا فشل mapping؛ في التطوير يسجل `taxonomy_mapping_error` ويُراجع مصدر البيانات.

الجنس

لا تعتمد على index ترتيب `texts.js` . استخدم key/`gender` من المرحلة الأولى، واظهر labels من `texts.js` فقط. اختبر:

- رجالي فقط.
- نسائي فقط.
- للجنسين فقط.
- عطر له occasion/note لكنه لا يطابق gender لا يظهر.

المناسبات

اختبر كل القيم الخمس: نهاري، ليلي، عمل، موعد، هدية. عطر مرتبط بأكثر من مناسبة يجب أن يظهر في كل فلتر مناسب له.

النوتات

اختبر عنبري، عود، خشبي، حمضي، هيل. ثبّت التسمية الصحيحة لـ `حمضي` /Citrus أو migration key موحد؛ لا تسمح بأن يؤدي اختلاف تهجئة عربي إلى نتيجة فارغة بلا warning.

6. مصفوفة قبول الظهور

استخدم fixture من المرحلة الأولى:

الحالة	المتوقع
بلا متجر، A=100 وB=120	يظهر بسعر 100 ومن متجر A
متجر B	لا يظهر لأن A هو صاحب العرض الأساسي
متجر C بلا عرض	لا يظهر
gender=men + occasion=work	يظهر فقط من يطابق الاثنين
occasion=work + note=oud	تقاطع حقيقي، لا اتحاد
size=50	لا يعرض 100 لنفس العطر
min=110 في الوضع العام	يستبعد السعر 100
min=110 داخل B	لا يظهر المنتج إذا لم يكن B صاحب العرض الأساسي
Add من A	الكارت والطلب يحملان A store_product_id والسعر 100
فتح الكارت بعد refresh	لا يتغير العرض أو السعر إلى عرض ثانٍ
global offer اختفى بعد الإضافة	حالة مراجعة، لا نقل لمتجر آخر

7. اختبارات الواجهة المطلوبة

أضف أو وسّع اختبارات `catalog-filters-style` وملفات storefront:

1. فلتر متجر لا يعرض إلا المنتج الذي يكون المتجر صاحبه للعرض الأساسي.
2. سعر البطاقة في المتجر يطابق global offer نفسه.
3. السعر العام يظل global خارج سياق المتجر.
4. أكثر من filter في نفس الطلب يعمل AND.
5. تعدد values داخل dimension يعمل OR.
6. الفئات الثلاث للجنس لا تعتمد على ترتيب النصوص.
7. تغيير ترتيب أو ترجمة labels لا يغير النتائج.
8. UUID/key الخاص بكل category/occasion/note يظل ثابتًا في الطلب والنتيجة.
9. كل حجم يحفظ `storeProductId` الصحيح.

10. إضافة من بطاقة عامة ثم بطاقة متجر لا تدمج العروض خطأ.
11. cartGroups يفصل متجرين لنفس المنتج/الحجم.
12. sendStoreOrder يرسل storeProductId المرتبط باختيار العميل.
13. manual/auto sections تستخدم نفس السعر والسياق.
14. أسماء المتاجر العربية/الإنجليزية أو escaping لا تكسر الفلتر.
15. السعر ومعرف العرض في البطاقة والكارت والطلب متطابقون حرفيًا.
16. عرض غير متاح أو غير synced أو من متجر غير نشط لا يدخل النتيجة أو الكارت.
17. تغير العرض الأساسي بعد الإضافة ينتج تنبيهًا/حالة مراجعة ولا يغير المتجر صامتًا.
18. pagination متعددة الصفحات بترتيب ثابت لا تكرر أو تستبدل العرض الفائز.

8. التحقق اليدوي قبل التسليم

أنشئ بيانات اختبار مؤقتة أو fixture محليًا لعطر واحد:

- منتج `P1`، حجم `50ml`.
- Store A: 100، متاح، synced.
- Store B: 120، متاح، synced.
- Store C: 90، غير متاح أو unsynced.

تحقق بصريًا من:

- صفحة all تعرض A/100.
- فلتر B يعرض B/120.
- إضافة من all تنتج طلب A.
- إضافة من A تنتج طلب A، ولا يمكن الإضافة من B لأن المنتج لا يظهر هناك.
- فلتر gender + occasion + note لا يضيف منتجات خارج التقاطع.
- بطاقة brand لا تتأثر بآخر متجر تم فتحه.

9. معايير عدم قبول الحل

يرفض التسليم إذا وُجد أي مما يلي:

- تغيير `list()` فقط دون تعديل الكارت.
- استخدام اسم المتجر كمفتاح وحيد مع توفر `store_id`.
- عرض سعر B مع إرسال `storeProductId` لـ A.

- إعادة اختيار الأرخص العام عند render الكارت.
- استمرار index-based mapping للفلاتر الجديدة.
- اختبار يثبت وجود product فقط دون إثبات الحجم والعرض والسعر والمتجر.
- تعديل `cheapest_offers` ليصبح متجراً محدداً، فهذا يكسر القاعدة العامة.

10. تسليم المرحلة

يجب تسليم:

- diff الواجهة والكارت والسيكشنات.
- اختبارات جديدة ونتيجة `npm test`.
- نتيجة `npm run check:migrations` و`npm run build:cloudflare`.
- جدول يدوي يثبت حالات A/B السابقة.
- وصف توافق عناصر الكارت القديمة.
- عدم وجود console/debug output في الإنتاج.